



المركز الوطني
للجواد العربي
NATIONAL CENTER
FOR THE ARABIAN
HORSE



Data specification of digital studbook system 'STUDLIB'

Date Created: February 2, 2026

Last Updated: March 16, 2026

Version: 1.01

Data Dictionary - StudLib Database.....	5
Overview.....	5
Table Categories.....	5
Core Domain Tables.....	6
horse.....	6
country.....	16
city.....	17
book_type.....	17
horse_color.....	19
Ownership Tables.....	20
owner.....	20
horse_owner.....	22
horse_breeder.....	22
Breeding Tables.....	23
season.....	23
covering_agreement.....	24
covering_certificate.....	26
covering_result.....	29
Event Tables.....	30
event_type.....	30
event.....	31
event_new_owner / event_old_owner.....	33
Laboratory Tables.....	34
laboratory.....	34
laboratory_test.....	34
laboratory_test_dna_sample_type.....	37
Horse Marking Tables.....	38
Users & Authentication Tables.....	42
application_user.....	42
refresh_token.....	44
token_blacklist.....	45
File Storage.....	45
file.....	45
Other Tables.....	47
email_log.....	47
forbidden_name.....	48
studbook_generation_task.....	49
Audit Tables (Hibernate Envers).....	51

revision_info.....	51
Audit table pattern.....	52
Relationships Summary.....	52
Primary Relationships.....	52
Database Views.....	54
Business Rules and Data Validation Constraints.....	55
Breed System.....	55
Horse Registration Lifecycle.....	55
Horse Identity and Naming.....	56
Horse Sex Classification.....	56
Pedigree Integrity.....	57
Horse Color Validation.....	57
Ownership Rules.....	58
Breeding and Covering Rules.....	58
Country and Locality Rules.....	59
Laboratory Testing Rules.....	60
User Access Control.....	60
Authentication Token Management.....	61
Event System Rules.....	61
File Management Rules.....	61
Optimistic Locking.....	62
Audit Trail Completeness.....	62
Search and Fuzzy Matching.....	62
Script Locations.....	63
Deployment Order.....	63
Rollback Strategy.....	63
Schema Versioning.....	63
Hibernate & Envers Configuration.....	64
Base Entity.....	64
Entity-to-Table Mapping Overview.....	64
Cascade Rules and Orphan Removal.....	65
Fetch Strategies.....	66
Custom Hibernate Types and Converters.....	67
Envers Configuration.....	68
Performance & Index Strategy.....	69
Query Patterns Driving Index Design.....	69
Tables Expected to Grow Significantly.....	71
Known Performance-Sensitive Areas.....	71
Maintenance Considerations.....	71

Business Rule Enforcement Matrix.....	72
Known Technical Limitations & Scaling Considerations.....	77
Schema Constraints That May Require Refactoring at Scale.....	77
Anticipated Growth Considerations.....	77

Data Dictionary - StudLib Database

Overview

StudLib is a horse studbook management system for tracking Arabian and Thoroughbred horses, their pedigrees, ownership, breeding activities, laboratory testing, and registration. The database uses PostgreSQL with Hibernate Envers for audit trails.

Table Categories

Category	Tables	Purpose
Core Domain	horse, horse_color, book_type, country, city	Horse registry and reference data
Ownership	owner, horse_owner, horse_breeder	Tracking horse ownership and breeders
Breeding	covering_agreement, covering_certificate, covering_result, season	Breeding season management
Events	event, event_type, event_new_owner, event_old_owner	Lifecycle event tracking
Laboratory	laboratory, laboratory_test, laboratory_test_dna_sample_type	DNA/parentage testing
Horse Markings	horse_face_whorl, horse_stripe, horse_star_, horse_hoof_, horse_white_marks_*, etc.	Physical identification markings
Users & Auth	application_user, refresh_token, token_blacklist	Authentication and authorization
Audit	*_aud tables, revision_info	Change history (Hibernate Envers)

Category	Tables	Purpose
Files	file	Document and image storage
Other	email_log, forbidden_name, studbook_generation_task	Supporting features

Core Domain Tables

horse

Central table of the system. Stores all registered horses.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	Record creation timestamp
modification_date	timestamp(6)	YES	Last modification timestamp
version	integer	NO	Optimistic locking version
name	varchar(255)	NO	Official registered horse name as it appears in the studbook
name_suffix	varchar(10)	YES	Disambiguation suffix appended to the name (e.g. country code or number) when multiple horses share the same name
local_name	varchar(255)	YES	Horse name in the local language/script,

Column	Type	Nullable	Description
			used in countries with non-Latin alphabets
breed	varchar(255)	NO	Horse breed: ARABIAN (purebred Arabian) or THOROUGHBRED . Determines which studbook, color palette, and book types apply
sex	varchar(255)	NO	Sex/reproductive classification: MALE (young unclassified male), FEMALE (young unclassified female), STALLION (approved breeding male), BROODMARE (breeding female), GELDING (castrated male), CRYPTORCHID (both testes undescended), MONORCHID (one testis undescended)
status	varchar(255)	NO	Registration workflow status. Progresses through: NEW_FOAL / NEW_IMPORTED -> WAITING_FOR_MARKING_DATA -> WAITING_FOR_LABORATORY -> LABORATORY_DATA_ENTERED -> REGISTER_IN_STUDBOOK . Terminal states: REJECTED ,

Column	Type	Nullable	Description
			EXTERNAL (foreign horse not fully registered locally)
horse_classification	varchar(255)	YES	Discipline/use classification: RACING, JUMPING, BEAUTY (show horse), UNKNOWN
uuid	uuid	NO	Globally unique identifier used in external integrations and API references, assigned at creation
ueln	varchar(15)	YES	Universal Equine Life Number - international 15-character identification code per WBFSH standard (e.g. "616002W00123456")
registry_id	varchar(255)	YES	Studbook registry number assigned upon registration in the local studbook (e.g. "AH-1234")
previous_registry_id	varchar(255)	YES	Registry number from the horse's previous studbook, applicable for imported horses
transponder_code	varchar(15)	YES	ISO 11784/11785 microchip identification number implanted in the horse

Column	Type	Nullable	Description
second_transponder_code	varchar(255)	YES	Secondary microchip number, used when a horse has been re-chipped or carries two transponders
transponder_site	varchar(255)	YES	Anatomical location where the microchip was implanted (e.g. left side of neck)
blood_code	varchar(255)	YES	Blood typing/group code used for historical identification before DNA profiling
strain	varchar(255)	YES	Arabian horse strain/family line (e.g. Kuhailan, Saqlawi, Dahman). Only relevant for Arabian breed
date_of_birth	date	YES	Foaling date
date_of_death	date	YES	Date of death, set when a DEATH event is recorded
date_of_registration	date	YES	Date when the horse was officially registered in the studbook (status changed to REGISTER_IN_STUDBOOK)
date_of_importing	date	YES	Date when the horse was imported into the country, set during

Column	Type	Nullable	Description
			IMPORT event processing
date_of_exporting	date	YES	Date when the horse was exported from the country, set during EXPORT event processing
date_of_control	date	YES	Date of the physical signalement examination (identification of markings, measurements, and microchip verification)
date_of_declaration	date	YES	Date when the foal was officially declared by the breeder/owner to the studbook authority
dna_sample_exists	boolean	YES	Indicates whether a biological sample (hair, blood, etc.) has been collected for DNA analysis (default false)
imported_from_file	boolean	YES	Indicates whether this horse record was created via bulk file import rather than manual entry (default false)
generated_registry_id	boolean	YES	Indicates the registry_id was automatically generated by the system rather than

Column	Type	Nullable	Description
			manually assigned (default false)
book_number	integer	YES	Volume number in the current studbook where this horse is registered
book_page	integer	YES	Page number within the studbook volume
book_appendix_number	integer	YES	Appendix number if the horse is registered in a studbook appendix rather than the main volume
previous_book_number	integer	YES	Volume number in the previous studbook (before transfer/import)
previous_book_page	integer	YES	Page number in the previous studbook
previous_book_appendix_number	integer	YES	Appendix number in the previous studbook
notes	text	YES	Free-text notes and remarks about the horse, visible to editors
legacy_registry_id	varchar(255)	YES	Registry ID carried over from the legacy/old studbook system during data migration

Column	Type	Nullable	Description
legacy_ueln	varchar(15)	YES	UELN from the legacy system, preserved for traceability during migration
face_whorl_other	varchar(255)	YES	Free-text description when face whorl marking is classified as OTHER in the horse_face_whorl collection
hoof_lf_other, hoof_lh_other, hoof_rf_other, hoof_rh_other	varchar(255)	YES	Free-text description when the respective hoof color is classified as OTHER (LF=Left Front, LH=Left Hind, RF=Right Front, RH=Right Hind)
mark_positions_other , marks_other	varchar(255)	YES	Free-text description for body marks/brands and their positions when classified as OTHER
snip_and_flesh_mark _other	varchar(255)	YES	Free-text description when the snip (white mark on muzzle/nostrils) or flesh mark is classified as OTHER
spot_on_coronet_lf_o ther, spot_on_coronet_lh_ other, spot_on_coronet_rf_ other,	varchar(255)	YES	Free-text description when the coronet (hoof band) spot is classified as OTHER for the respective leg

Column	Type	Nullable	Description
spot_on_coronet_rh_other			
star_position_other, star_rel_to_whorl_other, star_size_other	varchar(255)	YES	Free-text descriptions for forehead star marking when position, relation to whorl, or size is classified as OTHER
stripe_other	varchar(255)	YES	Free-text description when the facial stripe (white mark running down the face) is classified as OTHER
under_main_whorls_left_other, under_main_whorls_right_other	varchar(255)	YES	Free-text description when mane whorls (hair growth patterns under the mane) are classified as OTHER for the respective side
under_neck_whorls_other	varchar(255)	YES	Free-text description when neck/throat whorls are classified as OTHER
white_marks_lf_other, white_marks_lh_other, white_marks_rf_other, white_marks_rh_other	varchar(255)	YES	Free-text description when white leg markings (socks, stockings, etc.) are classified as OTHER for the respective leg
Foreign Keys			
father_id	bigint	YES	FK -> horse. The sire (father) of this horse,

Column	Type	Nullable	Description
			forming the paternal pedigree line
mother_id	bigint	YES	FK -> horse. The dam (mother) of this horse, forming the maternal pedigree line
birthplace_country_id	bigint	YES	FK -> country. Country where the horse was born; determines LOCAL vs IMPORTED origin
import_country_id	bigint	YES	FK -> country. Country from which the horse was imported; set during IMPORT event
export_country_id	bigint	YES	FK -> country. Destination country for exported horses; set during EXPORT event
book_type_id	bigint	YES	FK -> book_type. The studbook type/section where the horse is currently registered
previous_book_type_id	bigint	YES	FK -> book_type. Studbook type from the horse's previous registration (before import/transfer)
color_id	bigint	YES	FK -> horse_color. The horse's official coat color as

Column	Type	Nullable	Description
			recorded during signalement
controlled_by_id	bigint	YES	FK -> application_user. The veterinarian or qualified person who performed the signalement/identification examination
covering_result_id	bigint	YES	FK -> covering_result. Links the horse (as a foal) to the covering result that produced it
dna_card_id	bigint	YES	FK -> file (1:1). Uploaded scan/photo of the physical DNA sample card
export_certificate_id	bigint	YES	FK -> file (1:1). Uploaded export certificate document issued when the horse leaves the country
passport_id	bigint	YES	FK -> file (1:1). Uploaded equine passport document (official identification paper)

Self-referential relationships: *father_id* and *mother_id* reference back to *horse*, forming a pedigree tree.

country

Reference table for countries, used across horse origins, imports, exports, and owner addresses.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	
name	varchar(255)	NO	Full country name in English (unique). Used in UI dropdowns and on printed documents
alpha2	varchar(2)	NO	ISO 3166-1 alpha-2 code, e.g. "LY" for Libya (unique). Used in UELN generation and international data exchange
alpha3	varchar(3)	NO	ISO 3166-1 alpha-3 code, e.g. "LBY" for Libya (unique). Displayed on passports and export certificates
code	varchar(3)	NO	ISO 3166-1 numeric country code, e.g. "434" for Libya (unique)
local	boolean	NO	Flags the home country where this studbook authority operates. Exactly one

Column	Type	Nullable	Description
			country may be <code>true</code> (enforced by partial unique index). Used to determine horse origin: if <code>birthplace_country.local = true</code> , the horse is LOCAL; otherwise IMPORTED

Partial unique index: `unique_local_country` ensures only one country can be marked as `local = true`.

city

Simple lookup table for city names.

Column	Type	Nullable	Description
name	varchar(255)	NO	City name (primary key). Used as a reference/autocomplete list for city fields in owner addresses

book_type

Studbook types/volumes for horse registration.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	

Column	Type	Nullable	Description
modification_date	timestamp(6)	YES	
version	integer	NO	
code	varchar(128)	YES	Studbook code/abbreviation used on certificates and in registry IDs (e.g. "AHAR" for Arabian Horse Authority Register)
authority	varchar(255)	YES	Full name of the studbook issuing authority/organization (e.g. "World Arabian Horse Organization")
horse_breed	varchar(255)	NO	Breed this book type applies to: ARABIAN or THOROUGHBRED . Each breed has its own set of studbook types
libyan	boolean	YES	Marks this as a Libyan national studbook type (default false). Used to distinguish domestic vs foreign studbooks
country_id	bigint	YES	FK -> country. The country whose studbook authority issues this book type

horse_color

Reference table for horse coat colors, breed-specific.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	
name	varchar(255)	NO	Full coat color name as it appears on official documents (e.g. "Bay", "Chestnut", "Grey"). Unique per breed
short_name	varchar(255)	NO	Abbreviated color code used in lists and compact views (e.g. "b.", "ch.", "gr."). Unique per breed
horse_breed	varchar(255)	NO	Breed this color applies to: ARABIAN or THOROUGHBRED . Color palettes differ between breeds
validation_rule	varchar(255)	YES	Inheritance validation rule for this color: BOTH_PARENTS means both sire and dam must carry the color gene for it to be valid; AT_LEAST_ONE_PARENT means at least one parent must

Column	Type	Nullable	Description
			carry it. Used during foal registration to validate color plausibility

Unique constraints: (horse_breed, name) and (horse_breed, short_name).

Ownership Tables

owner

People or businesses that own or breed horses.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	
full_name	varchar(255)	NO	Full legal name of the person or business entity. Part of the uniqueness constraint (full_name, country_id, city)
suffix	varchar(255)	YES	Globally unique short identifier used to disambiguate owners with similar names (e.g. on printed certificates). Searchable via trigram index

Column	Type	Nullable	Description
business	boolean	YES	<code>true</code> if the owner is a legal entity (company, farm, syndicate); <code>false</code> for individuals (default false). Determines which ID fields are relevant
business_number	varchar(128)	YES	Business/company registration number. Applicable when <code>business = true</code> . Searchable via trigram index
national_id_number	varchar(128)	YES	Personal national ID or passport number. Applicable when <code>business = false</code> . Searchable via trigram index
farm_name	varchar(255)	YES	Name of the farm, stud, or stable operated by this owner
phone	varchar(255)	YES	Contact phone number
note	text	YES	Internal notes about this owner, not printed on official documents
street	varchar(255)	YES	Street address line
city	varchar(255)	YES	City name. Part of the uniqueness constraint
postal_code	varchar(255)	YES	Postal/ZIP code

Column	Type	Nullable	Description
region	varchar(255)	YES	Region, state, or province
latitude	double precision	YES	GPS latitude of the farm/residence, used for map display
longitude	double precision	YES	GPS longitude of the farm/residence, used for map display
country_id	bigint	YES	FK -> country. Owner's country of residence. Part of the uniqueness constraint

Unique constraint: (full_name, country_id, city). **Trigram indexes:** on full_name, business_number, national_id_number, suffix for fuzzy search.

horse_owner

Many-to-many join table tracking the current owner(s) of a horse. A horse can have multiple co-owners (e.g. syndicate ownership). Updated via CHANGE_OWNER events.

Column	Type	Nullable	Description
horse_id	bigint	NO	FK -> horse. The owned horse (composite PK)
owner_id	bigint	NO	FK -> owner. One of the horse's current owners (composite PK)

horse_breeder

Many-to-many join table recording the breeder(s) of a horse. Breeders are the owner(s) of the dam at the time of foaling. Set during NEW_FOAL registration and typically does not change.

Column	Type	Nullable	Description
horse_id	bigint	NO	FK -> horse. The bred horse (composite PK)
owner_id	bigint	NO	FK -> owner. One of the horse's breeders (composite PK)

Breeding Tables

season

Breeding seasons by year.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	
year	integer	NO	Calendar year of the breeding season (unique). Used to group covering agreements and certificates
start_date	date	YES	Official opening date of the breeding season, after which coverings can be registered

Column	Type	Nullable	Description
end_date	date	YES	Official closing date of the breeding season, after which no new coverings should be recorded
active	boolean	YES	Whether this is the currently active breeding season. Only one season should be active at a time; controls which season is pre-selected in covering forms

covering_agreement

Stallion breeding agreements per season.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	
type	varchar(255)	NO	Agreement type defining the terms of stallion access: PUBLIC (open to any mare owner), PRIVATE (restricted to specific mare owners), PERMANENT (multi-season),

Column	Type	Nullable	Description
			TEMPORARY (single-season), DECLINED (application rejected), PENDING (awaiting approval). Unique per (type, stallion, season)
agreement_number_min	integer	YES	Start of the allocated certificate number range for this agreement (e.g. 1). Each mare covering within this agreement is assigned a sequential number in this range
agreement_number_max	integer	YES	End of the allocated certificate number range (e.g. 50). Defines the maximum number of mares the stallion can cover under this agreement
location	varchar(255)	YES	Physical location/address where the coverings take place (stud farm name or address)
notes	varchar(1000)	YES	Additional notes about the agreement terms, conditions, or special arrangements
season_id	bigint	NO	FK -> season. The breeding season this agreement belongs to

Column	Type	Nullable	Description
stallion_id	bigint	NO	FK -> horse. The stallion authorized to breed under this agreement

Unique constraint: (type, stallion_id, season_id).

covering_certificate

Individual mare covering records within an agreement.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	
agreement_number	integer	YES	Sequential certificate number within the agreement's allocated range (between agreement_number_min and agreement_number_max)
agreement_number_denormalized	varchar(7)	YES	Zero-padded 6-digit string representation of agreement_number (e.g. "000042"). Must satisfy CHECK: length = 6 and value =

Column	Type	Nullable	Description
			lpad(agreement_number, 6, '0'). Used in display and combined with season year for the full certificate reference (e.g. "2025 000042")
covering_type	varchar(255)	YES	Method of breeding: NATURAL (live cover) or ARTIFICIAL_INSEMINATION (AI)
first_covering_date	date	YES	Date of the first mating/insemination attempt in this covering period
last_covering_date	date	YES	Date of the last mating/insemination attempt; may equal first_covering_date for a single service
location	varchar(255)	YES	Specific location where this covering took place; may differ from the agreement-level location
semen_origin	varchar(255)	YES	Origin of semen for AI: LOCAL (collected domestically) or IMPORTED (from abroad). Only relevant when covering_type = ARTIFICIAL_INSEMINATION

Column	Type	Nullable	Description
semen_type	varchar(255)	YES	Type of semen used for AI: FRESH (used immediately) or FROZEN (stored/shipped). Only relevant when covering_type = ARTIFICIAL_INSEMINATION
covering_agreement_id	bigint	YES	FK -> covering_agreement. Links to the local stallion's covering agreement. Mutually exclusive with external_stallion_id
external_stallion_id	bigint	YES	FK -> horse. References a foreign/external stallion not covered by a local agreement (e.g. imported semen or mare sent abroad). Mutually exclusive with covering_agreement_id
mare_id	bigint	NO	FK -> horse. The mare being covered under this certificate

Check constraint: Exactly one of **covering_agreement_id** or **external_stallion_id** must be set.

covering_result

Outcome of a covering (one per certificate).

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	
covering_result_type	varchar(255)	NO	Outcome of the covering: BORN_FOAL (single live foal), BORN_TWINS (twin foals), DEATH (mare or foal died), ABORTED (pregnancy terminated), NOT_COVERED (mare was not successfully bred). Determines whether a foal record is created
date	date	YES	Date when the outcome occurred (birth date, abortion date, etc.)
covering_certificate_id	bigint	NO	FK -> covering_certificate (unique). One-to-one relationship: each certificate has at most one recorded result

Event Tables

event_type

Reference table for types of horse lifecycle events.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	
type	varchar(255)	YES	Event type enum: CHANGE_OWNER (ownership transfer), FOAL_ABORTION (pregnancy loss), NEW_FOAL (foal birth registered), DEATH (horse died), IMPORT (horse entered the country), EXPORT (horse left the country), GELDING (castration performed), COVERED (mare was bred)
description	varchar(255)	YES	Human-readable label shown in the UI for this event type (e.g. "Ownership Change", "New Foal Registration")
selectable	boolean	NO	Controls whether this event type appears in the UI event creation

Column	Type	Nullable	Description
			dropdown (default true). Set to false to deprecate an event type without deleting it or its historical data

event

Horse lifecycle events.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	
date	date	YES	Date when the event occurred (e.g. sale date, death date, import date)
description	varchar(255)	YES	Free-text description or additional details about this specific event instance
additional_information	varchar(255)	YES	Supplementary notes, e.g. cause of death, import permit number, or other event-specific details
cover	boolean	YES	For COVERED events: indicates whether the mare was successfully

Column	Type	Nullable	Description
			covered during this event
pregnant	boolean	YES	For COVERED events: indicates whether the mare is confirmed pregnant following the covering
twins	boolean	YES	For COVERED/NEW_FOAL events: indicates a twin pregnancy or twin birth
horse_sex	smallint	YES	For NEW_FOAL events: sex of the newborn foal as a numeric enum (0-6 mapping to the horse sex values: MALE=0, FEMALE=1, STALLION=2, BROODMARE=3, GELDING=4, CRYPTORCHID=5, MONORCHID=6)
type_id	bigint	YES	FK -> event_type. The category of this event (DEATH, IMPORT, EXPORT, etc.)
horse_primary_id	bigint	YES	FK -> horse. The main subject horse of the event (e.g. the horse that died, was imported, or the mare that was covered)

Column	Type	Nullable	Description
horse_secondary_id	bigint	YES	FK -> horse. Secondary horse involved in the event (e.g. the stallion in a COVERED event, or the sire in a NEW_FOAL event)
horse_offspring_id	bigint	YES	FK -> horse. The foal produced by the event, applicable for NEW_FOAL events. Links the birth event to the newly created horse record
country_id	bigint	YES	FK -> country. Relevant country for IMPORT/EXPORT events (origin or destination country)

event_new_owner / event_old_owner

Many-to-many join tables linking CHANGE_OWNER events to the involved owners.

event_new_owner records the owner(s) receiving the horse after the transfer.

event_old_owner records the owner(s) giving up the horse in the transfer.

Column	Type	Nullable	Description
event_id	bigint	NO	FK -> event. The ownership change event (composite PK)
owner_id	bigint	NO	FK -> owner. The new or previous owner involved in the

Column	Type	Nullable	Description
			transfer (composite PK)

Laboratory Tables

laboratory

DNA testing laboratories.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	
name	varchar(255)	YES	Full name of the DNA testing laboratory (e.g. "Veterinary Genetics Laboratory"). Referenced by lab tests and lab user accounts

laboratory_test

DNA parentage verification and genotyping tests.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key

Column	Type	Nullable	Description
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	
test_type	varchar(255)	YES	Type of test requested: FULL_PARENAGE_VERIFICATION (verify both sire and dam), IDENTITY_VERIFICATION (confirm horse identity against existing genotype), GENOTYPE_ONLY (generate DNA profile without parentage check), INVESTIGATION (special investigation for disputed parentage or identity)
test_priority	varchar(255)	YES	Processing priority: URGENT (expedited processing, e.g. for export or sale deadlines) or NORMAL (standard turnaround)
laboratory_status	varchar(255)	YES	Current status of the test in the lab workflow: NOT_STARTED (sample not yet processed), WAITING_FOR_RESULT (analysis in progress), PARENAGE_VERIFY

Column	Type	Nullable	Description
			ED (both parents confirmed), PARENTAGE_NOT_VERIFIED (parentage mismatch), PARENTAGE_VERIFIED_WITH_DAM_NOT_SIRE / ..._WITH_SIRE_NOT_DAM (partial verification), DAM_NOT_GENOTYPED / SIRE_NOT_GENOTYPED / DAM_AND_SIRE_NOT_GENOTYPED (parent samples missing), various *_NEEDS_RECOLLECTION statuses (sample quality insufficient), OTHER
batch_ref_number	varchar(255)	YES	Laboratory batch reference number grouping multiple samples shipped together
dna_reference_number	varchar(255)	YES	Unique DNA reference number assigned by the laboratory for this horse's genetic profile
shipment_date	date	YES	Date the biological samples were shipped from the

Column	Type	Nullable	Description
			studbook authority to the laboratory
shipment_received_date	date	YES	Date the laboratory confirmed receipt of the samples
notes	varchar(255)	YES	Notes about the test, e.g. special instructions, issues with samples, or lab comments
horse_id	bigint	YES	FK -> horse. The horse being tested
laboratory_id	bigint	YES	FK -> laboratory. The laboratory performing the test
entered_by_id	bigint	YES	FK -> application_user. The user (typically with LAB_USER role) who entered or last updated the test results

laboratory_test_dna_sample_type

DNA sample types per test (collection table).

Column	Type	Nullable	Description
laboratory_test_id	bigint	NO	FK -> laboratory_test. The test these samples are associated with

Column	Type	Nullable	Description
dna_sample_type	varchar(255)	YES	Type of biological sample collected: HAIR (pulled hair roots - most common), BLOOD (blood sample), BONE (bone tissue - used for deceased horses), FTA_CARD (Flinders Technology Associates card - dried blood/tissue on filter paper for storage and transport). Multiple sample types can be associated with one test

Horse Marking Tables

These are collection tables (element collections) storing multi-valued physical identification markings for each horse. All reference **horse(id)** via **horse_id** FK.

Table	Value Column	Allowed Values
horse_face_whorl	face_whorl	NONE, ABOVE_EYES, AT_EYE_LEVEL, BELOW_EYE_LEVEL, TWO_ABOVE, TWO_AT, TWO_BELOW, TWO_ONE_ABOVE_ONE_BELOW, TWO_SUPERPOSED, TWO_JUXTAPOSED, TWO_DIAGONAL, NASAL_BONE, OTHER

Table	Value Column	Allowed Values
horse_star_position	star_position	NONE, MEDIAN, LEFT, RIGHT, VERTICAL, DIAGONAL, IRREGULAR, OTHER
horse_star_size	star_size	FEW_HAIRS, SMALL_STAR, STAR, LARGE_STAR, MIXED_SMALL_STAR, MIXED_STAR, LARGE_MIXED_STAR, STAINED, BORDERED, OTHER
horse_star_rel_to_whorl	star_rel_to_whorl	INCLUDING, ABOVE, BELOW, LEFT_TO, RIGHT_TO, TANGENT, OTHER
horse_stripe	stripe	NONE, FEW_HAIRS, FINE_STRIPE, REGULAR_STRIPE, LARGE_STRIPE, MIXED_FINE, MIXED, LARGE_MIXED_BLAZE, STAINED_BLAZE, BORDERED_BLAZE, INTERRUPTED, WHITE_FACE, OTHER
horse_snip_and_flesh_mark	snip_and_flesh_mark	NONE, BETWEEN_NOSTRILS, LEFT_NOSTRIL, RIGHT_NOSTRIL, BOTH_NOSTRILS, NOSTRILS_UPPER_LIP, NOSTRILS_LOWER_LIP, STRIP_NOSTRILS_LIPS, STRIP_BOTH_NOSTRILS_LIPS, FB_NOSTRILS_LIPS_CHIN, OTHER

Table	Value Column	Allowed Values
horse_marks	marks	NONE, HOT_IRON, NITROGEN, HEAD, NECK_CHEST, BODY, FORELIMBS, HINDLIMBS, HINDQUARTERS, MULTIPLE, OTHER
horse_mark_positions	mark_positions	RIGHT, LEFT, HEAD, NECK, BODY, FRONT_LEG, HIND_LEG
horse_hoof_lf	hoof_lf	WHITE, DARK, MIXED, OTHER
horse_hoof_lh	hoof_lh	WHITE, DARK, MIXED, OTHER
horse_hoof_rf	hoof_rf	WHITE, DARK, MIXED, OTHER
horse_hoof_rh	hoof_rh	WHITE, DARK, MIXED, OTHER
horse_spot_on_coronet_lf	spot_on_coronet_lf	BLACK_CORONET, CHESTNUT_CORONET, OTHER
horse_spot_on_coronet_lh	spot_on_coronet_lh	BLACK_CORONET, CHESTNUT_CORONET, OTHER
horse_spot_on_coronet_rf	spot_on_coronet_rf	BLACK_CORONET, CHESTNUT_CORONET, OTHER
horse_spot_on_coronet_rh	spot_on_coronet_rh	BLACK_CORONET, CHESTNUT_CORONET, OTHER
horse_white_marks_lf	white_marks_lf	NONE, CORONET_WHITE, WHITE_SPOT_CORONET, HALF_PASTERN_WHITE, FULL_PASTERN_WHITE,

Table	Value Column	Allowed Values
		FETLOCK_WHITE, THIRD_CANNON_WHITE, HALF_CANNON_WHITE, TO_THIRD_CANNON_WHITE, UP_TO_CANNON_WHITE, WHITE_TO_KNEE_HOCK, ABOVE_KNEE_HOCK, OTHER
horse_white_marks_lh	white_marks_lh	<i>(same as above)</i>
horse_white_marks_rf	white_marks_rf	<i>(same as above)</i>
horse_white_marks_rh	white_marks_rh	<i>(same as above)</i>
horse_under_main_whorls_left	under_main_whorls_left	NONE, POLL, BEHIND_POLL, UPPER_NECK, MID_NECK, LOWER_NECK, TWO_UPPER, TWO_LOWER, ONE_UPPER_ONE_LOWER, MORE_THAN_TWO, OTHER
horse_under_main_whorls_right	under_main_whorls_right	<i>(same as above)</i>
horse_under_neck_whorls	under_neck_whorls	NONE, THROAT, BELOW_THROAT, UPPER_THIRD, MID_TRACHEA, LOWER_THIRD, TWO_UPPER, TWO_MID, TWO_LOWER, ONE_MID_ONE_OTHER, ONE_UPPER_ONE_LOWER, OTHER

Note: LF = Left Front, LH = Left Hind, RF = Right Front, RH = Right Hind.

Users & Authentication Tables

application_user

System users with role-based access.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	
email	varchar(128)	NO	Login email address. Part of unique constraint (email, breed) - same person may have separate accounts per breed
password	varchar(128)	NO	BCrypt-hashed password
full_name	varchar(256)	YES	User's display name shown in the UI and recorded in audit trails
system_role	varchar(24)	NO	Access level defining what actions the user can perform: LAB_USER (can enter lab test results only), VIEWER (read-only access), FIRST_LEVEL_EDITOR (basic data entry), SECOND_LEVEL_EDITOR (can approve

Column	Type	Nullable	Description
			registrations), THIRD_LEVEL_EDITOR (can manage covering agreements), FOURTH_LEVEL_EDITOR (senior editor), ADMIN (full system access including user management)
breed	varchar(24)	NO	The breed context this account operates in. Users see only horses and data for their assigned breed. A person needing access to both breeds requires two accounts
deleted	boolean	NO	Soft delete flag (default false). Deleted users cannot log in but their data is preserved for audit trail integrity
reset_password_token	varchar(64)	YES	One-time token sent via email for password reset flow. Cleared after use
laboratory_id	bigint	YES	FK -> laboratory. Associates LAB_USER role accounts with their laboratory; determines which lab test results they can enter

Unique constraint: (email, breed).

refresh_token

JWT refresh tokens for user sessions.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
token	varchar(500)	NO	The refresh token string value (unique). Presented by the client to obtain a new access token without re-authentication
created_at	timestampz	NO	Timestamp when the refresh token was issued
expiry_date	timestampz	NO	Timestamp after which the refresh token is no longer valid and cannot be used to obtain new access tokens
revoked	boolean	NO	Set to true when the token is explicitly revoked (e.g. on logout or security concern). Revoked tokens are rejected even before expiry
user_id	bigint	NO	FK -> application_user. The user this refresh token was issued to

token_blacklist

Blacklisted JWT access tokens.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
token	varchar(1000)	NO	The JWT access token string that has been invalidated (unique). Checked on each authenticated request to reject revoked tokens
blacklisted_at	timestampz	NO	Timestamp when the token was added to the blacklist (e.g. on user logout or forced session termination)
expiry_date	timestampz	NO	Original expiration date of the token. Blacklist entries can be safely cleaned up after this date since expired tokens are rejected regardless

File Storage

file

Stores metadata for uploaded files (passports, DNA cards, export certificates, horse photos).

Column	Type	Nullable	Description
file_type	varchar(31)	NO	Single-table inheritance discriminator identifying the file subtype (e.g. passport document, DNA card, export certificate, horse photo)
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	
file_name	varchar(255)	NO	Original file name as uploaded by the user (e.g. "passport_scan.pdf")
file_path	varchar(255)	NO	Internal storage path on the file system or object storage (unique). Used to retrieve the file content
file_size	bigint	NO	File size in bytes. Used for storage management and download progress
mime_type	varchar(255)	NO	MIME content type (e.g. "application/pdf", "image/jpeg"). Used for Content-Type headers when serving the file

Column	Type	Nullable	Description
document_type	smallint	YES	Categorization of horse documents: 0 = identification document, 1 = certificate, 2 = other. Determines where the document appears in the horse's file section
is_avatar	boolean	YES	When true , this image file is used as the horse's profile photo/thumbnail displayed in lists and detail pages
horse_id	bigint	YES	FK -> horse. The horse this file is associated with. NULL for files not linked to a specific horse

Other Tables

email_log

Log of sent email notifications.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	

Column	Type	Nullable	Description
recipient	varchar(255)	YES	Email address of the recipient
subject	varchar(255)	YES	Email subject line
content	text	YES	Full email body content (HTML or plain text)
sent_at	timestamp(6)	YES	Timestamp when the email was actually sent (may differ from creation_date if sending was delayed or retried)

forbidden_name

Names that cannot be used for horse registration.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	
name	varchar(255)	NO	Horse name that is prohibited from being used in new registrations (e.g. reserved names, offensive terms, names of famous historical horses that cannot be reused)

studbook_generation_task

Tracks async studbook PDF generation jobs.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Primary key
creation_date	timestamp(6)	YES	
modification_date	timestamp(6)	YES	
version	integer	NO	
breed	varchar(255)	NO	Breed of the studbook being generated: ARABIAN or THOROUGHBRED
year_from	integer	NO	First registration year to include in the generated studbook volume
year_to	integer	NO	Last registration year to include in the generated studbook volume
volume	integer	NO	Volume/tome number of the studbook being generated (used in the document title and metadata)
status	varchar(255)	NO	Task lifecycle status: IN_PROGRESS (generation currently running), COMPLETED (PDF successfully generated and available for download), FAILED

Column	Type	Nullable	Description
			(generation failed - see error_message), CANCELLED (user cancelled the generation)
created_at	timestamp(6)	NO	Timestamp when the generation task was queued
started_at	timestamp(6)	YES	Timestamp when the background worker began processing this task
completed_at	timestamp(6)	YES	Timestamp when the task finished (whether successfully or with failure)
file_url	varchar(255)	YES	URL/path to the generated studbook PDF file. Set when status = COMPLETED; used to serve the download
error_message	text	YES	Technical error details when status = FAILED. Used by admins for troubleshooting
created_by_id	bigint	NO	FK -> application_user. The user who requested the studbook generation

Audit Tables (Hibernate Envers)

All `*_aud` tables mirror their base table structure plus audit metadata. They track every change made to entities.

revision_info

Master audit revision table. Each database transaction that modifies audited entities creates one revision.

Column	Type	Nullable	Description
id	bigint (identity)	NO	Revision ID (primary key). Sequentially assigned; higher ID = later change. Referenced by all <code>*_aud</code> tables
revision_timestamp	bigint	YES	Epoch timestamp in milliseconds (Hibernate Envers default). Used internally for revision ordering
custom_timestamp	timestamp(6)	YES	Human-readable timestamp of the change. Indexed (DESC) for efficient history log queries with date range filters
username	varchar(255)	YES	Email/login of the user who made the change. Indexed for filtering history by user. Joined with <code>application_user</code> in the <code>vhistory_log</code> view

Column	Type	Nullable	Description
usern_fullname	varchar(255)	YES	Full display name of the user at the time of the change, denormalized for quick display without joining
ip_address	varchar(255)	YES	Client IP address from which the change was made, captured for security auditing

Audit table pattern

Each *_aud table has:

- All columns from the base table
- **rev** (bigint) - FK -> revision_info, part of composite PK
- **revtype** (smallint) - 0 = INSERT, 1 = UPDATE, 2 = DELETE
- Composite PK: (rev, id) for entity tables, or (rev, entity_id, collection_element) for collection tables

Audited entities: horse, country, book_type, horse_color, owner, event, event_type, covering_agreement, covering_certificate, covering_result, laboratory_test, season, and all horse marking/ownership collection tables.

Relationships Summary

Primary Relationships

```
None
horse
|-- father_id -----> horse (self-ref: sire)
|-- mother_id -----> horse (self-ref: dam)
|-- birthplace_country_id -> country
|-- import_country_id -----> country
|-- export_country_id -----> country
```

```

|-- book_type_id -----> book_type
|-- previous_book_type_id -> book_type
|-- color_id -----> horse_color
|-- covering_result_id ----> covering_result
|-- controlled_by_id -----> application_user
|-- dna_card_id -----> file (1:1)
|-- export_certificate_id -> file (1:1)
|-- passport_id -----> file (1:1)

horse_owner (M:N)
|-- horse_id -> horse
|-- owner_id -> owner

horse_breeder (M:N)
|-- horse_id -> horse
|-- owner_id -> owner

covering_agreement
|-- season_id ----> season
|-- stallion_id --> horse

covering_certificate
|-- covering_agreement_id -> covering_agreement
|-- external_stallion_id --> horse (XOR with agreement)
|-- mare_id -----> horse

covering_result
|-- covering_certificate_id -> covering_certificate (1:1)

event
|-- type_id -----> event_type
|-- horse_primary_id ---> horse
|-- horse_secondary_id -> horse
|-- horse_offspring_id -> horse
|-- country_id -----> country

event_new_owner / event_old_owner (M:N)
|-- event_id -> event
|-- owner_id -> owner

laboratory_test
|-- horse_id -----> horse
|-- laboratory_id --> laboratory
|-- entered_by_id --> application_user

application_user
|-- laboratory_id -> laboratory

```

```

refresh_token
  |-- user_id -> application_user

file
  |-- horse_id -> horse

book_type
  |-- country_id -> country

All *_aud tables
  |-- rev -> revision_info

```

Database Views

View	Purpose
vhorse	Denormalized horse list with color name, origin (LOCAL/IMPORTED), offspring count, and aggregated stallion/mare covering agreements
vhorse_event	Joins horse with its events and event types for event-based horse queries
vhistory_log	Comprehensive audit history view joining revision_info with all audited entity tables; used for the history/activity log UI
vstudbook_imported_horse_data	Imported horses with breeder names, sire/dam names, previous studbook info
vpassport_data	Complete passport data including 3-generation pedigree (sire/dam lines), owner/breeder names, color, studbook info, and lab status

Business Rules and Data Validation Constraints

Breed System

The system manages two distinct horse breeds, enforced throughout all breed-scoped entities:

Rule	Enforcement
Only ARABIAN and THOROUGHBRED breeds are allowed	CHECK constraints on horse.breed , book_type.horse_breed , horse_color.horse_breed , studbook_generation_task.breed
Horse colors are breed-specific	UNIQUE(horse_breed, name) and UNIQUE(horse_breed, short_name) on horse_color
Users are scoped to a breed	application_user.breed is NOT NULL; UNIQUE(email, breed) allows the same email for different breed contexts
Book types belong to a breed	book_type.horse_breed is NOT NULL

Horse Registration Lifecycle

Horses progress through a defined status workflow:

```
None
NEW_FOAL
  |
  v
WAITING_FOR_MARKING_DATA
  |
  v
WAITING_FOR_LABORATORY
  |
  v
LABORATORY_DATA_ENTERED
  |
  v
REGISTER_IN_STUDBOOK -----> (final registered state)

REJECTED -----> (terminal state)
```

EXTERNAL -----> (horses from other studbooks, not fully registered locally)

None
NEW_IMPORTED
|
v
REGISTER_IN_STUDBOOK -----> (final registered state)

REJECTED -----> (terminal state)

- `horse.status` is NOT NULL - every horse must have a defined status.
- `horse.uuid` is NOT NULL - every horse gets a unique external identifier at creation.
- `horse.name` and `horse.sex` are NOT NULL - basic identity is always required.
- Registration date (`date_of_registration`) is tracked for the `REGISTER_IN_STUDBOOK` transition, audited via `horse_aud` with index `idx_horse_aud_status_registration`.

Horse Identity and Naming

Rule	Enforcement
Horse names can be restricted	<code>forbidden_name</code> table holds names that must not be used for new registrations
Horse name suffix is limited to 10 characters	<code>horse.name_suffix</code> varchar(10)
UELN is a standardized 15-character code	<code>horse.ueln</code> varchar(15)
Transponder (microchip) code is max 15 characters	<code>horse.transponder_code</code> varchar(15)
Registry ID denormalized format must be zero-padded and match the numeric value	CHECK on <code>covering_certificate.agreement_number_denormalized</code> (see Breeding section)

Horse Sex Classification

The `horse.sex` field uses an extended classification beyond biological sex:

Value	Description
MALE	Young male, not yet classified
FEMALE	Young female, not yet classified
STALLION	Breeding male
BROODMARE	Breeding female
GELDING	Castrated male
CRYPTORCHID	Male with undescended testes (both)
MONORCHID	Male with one undescended testis

The **GELDING** event type transitions a horse's sex classification.

Pedigree Integrity

Rule	Enforcement
A horse has at most one sire and one dam	<code>horse.father_id</code> and <code>horse.mother_id</code> are single-valued FK references to <code>horse</code>
Pedigree is a directed acyclic graph	Self-referential FK on <code>horse</code> (no DB-level cycle prevention; application must enforce)
Pedigree is queryable to at least 3 generations	<code>vpassport_data</code> view joins sire/dam lines 3 levels deep (sire_sire_sire, dam_dam_dam, etc.)

Horse Color Validation

Rule	Enforcement
Color must match the horse's breed	<code>horse.color_id</code> FK -> <code>horse_color</code> , where <code>horse_color</code> has breed-specific entries
Color inheritance can be validated against parents	<code>horse_color.validation_rule: BOTH_PARENTS</code> (both parents must carry the

Rule	Enforcement
	color gene) or <code>AT_LEAST_ONE_PARENT</code> (at least one parent must carry it)

Ownership Rules

Rule	Enforcement
A horse can have multiple simultaneous owners	M:N via <code>horse_owner</code> join table
A horse can have multiple breeders	M:N via <code>horse_breeder</code> join table
Owner identity must be unique within a country and city	<code>UNIQUE(full_name, country_id, city)</code> on <code>owner</code>
Owner suffix is globally unique	<code>UNIQUE</code> on <code>owner.suffix</code>
Ownership changes are tracked as events	<code>event_type.type = 'CHANGE_OWNER'</code> with <code>event_old_owner</code> and <code>event_new_owner</code> recording before/after

Breeding and Covering Rules

Rule	Enforcement
A covering certificate must reference either an internal agreement OR an external stallion, never both	CHECK constraint <code>exactly_one_stallion_source_check:</code> <code>(external_stallion_id IS NOT NULL AND covering_agreement_id IS NULL)</code> OR <code>(external_stallion_id IS NULL AND covering_agreement_id IS NOT NULL)</code>
A stallion can have only one agreement of a given type per season	<code>UNIQUE(type, stallion_id, season_id)</code> on <code>covering_agreement</code>
Agreement numbers within a certificate must be formatted consistently	CHECK: <code>agreement_number_denormalized</code> must be exactly 6 characters and equal to <code>lpad(agreement_number, 6, '0')</code>

Rule	Enforcement
Certificate numbers must be unique per agreement	UNIQUE(agreement_number, covering_agreement_id, external_stallion_id) and UNIQUE(agreement_number_denormalized, covering_agreement_id, external_stallion_id) on <code>covering_certificate</code>
Each covering certificate has at most one result	<code>covering_result.covering_certificate_id</code> is UNIQUE
Covering result outcomes are enumerated	<code>BORN_FOAL</code> , <code>BORN_TWINS</code> , <code>DEATH</code> , <code>ABORTED</code> , <code>NOT_COVERED</code>
A season year must be unique	UNIQUE on <code>season.year</code>
Agreement types define the nature of the agreement	<code>PRIVATE</code> , <code>PUBLIC</code> , <code>PERMANENT</code> , <code>TEMPORARY</code> , <code>DECLINED</code> , <code>PENDING</code>
Covering method is tracked	<code>covering_type</code> : <code>NATURAL</code> or <code>ARTIFICIAL_INSEMINATION</code> ; <code>semen_origin</code> : <code>LOCAL</code> or <code>IMPORTED</code> ; <code>semen_type</code> : <code>FRESH</code> or <code>FROZEN</code>

Country and Locality Rules

Rule	Enforcement
Exactly one country can be marked as local	Partial unique index <code>unique_local_country</code> on <code>country(local)</code> WHERE (<code>local</code> = <code>true</code>)
Country codes follow ISO standards	<code>alpha2</code> varchar(2), <code>alpha3</code> varchar(3), <code>code</code> varchar(3), all NOT NULL and UNIQUE
A horse's origin is derived from birthplace country	<code>vhorse</code> view: if <code>country.local</code> = <code>true</code> then origin = <code>LOCAL</code> , else <code>IMPORTED</code>

Laboratory Testing Rules

Rule	Enforcement
Test types are enumerated	<code>FULL_PARENTAGE_VERIFICATION</code> , <code>IDENTITY_VERIFICATION</code> , <code>GENOTYPE_ONLY</code> , <code>INVESTIGATION</code>
Test priority levels	<code>URGENT</code> or <code>NORMAL</code>
Laboratory status tracks full parentage verification workflow	16 distinct statuses covering verified, not verified, genotyping gaps, and sample recollection needs
DNA samples can be of multiple types per test	Collection table <code>laboratory_test_dna_sample_type</code> with values <code>HAIR</code> , <code>BLOOD</code> , <code>BONE</code> , <code>FTA_CARD</code>
Tests are linked to the entering user	<code>laboratory_test.entered_by_id</code> FK -> <code>application_user</code>

User Access Control

Rule	Enforcement
Users have a hierarchical role system	7 roles in ascending privilege: <code>LAB_USER</code> , <code>VIEWER</code> , <code>FIRST_LEVEL_EDITOR</code> , <code>SECOND_LEVEL_EDITOR</code> , <code>THIRD_LEVEL_EDITOR</code> , <code>FOURTH_LEVEL_EDITOR</code> , <code>ADMIN</code>
Lab users are associated with a laboratory	<code>application_user.laboratory_id</code> FK -> <code>laboratory</code>
Users are soft-deleted, never hard-deleted	<code>application_user.deleted</code> boolean (default false)
A user account is unique per email + breed combination	<code>UNIQUE(email, breed)</code> - the same person can have separate accounts for Arabian and Thoroughbred
Password reset uses one-time tokens	<code>application_user.reset_password_token</code> <code>varchar(64)</code>

Authentication Token Management

Rule	Enforcement
Refresh tokens are unique	UNIQUE on <code>refresh_token.token</code>
Refresh tokens can be revoked	<code>refresh_token.revoked</code> boolean
Access tokens can be blacklisted (e.g. on logout)	<code>token_blacklist</code> table with unique token value
Blacklisted tokens store their expiry for automated cleanup	<code>token_blacklist.expiry_date</code>

Event System Rules

Rule	Enforcement
Event types are predefined	<code>CHANGE_OWNER</code> , <code>FOAL_ABORTION</code> , <code>NEW_FOAL</code> , <code>DEATH</code> , <code>IMPORT</code> , <code>EXPORT</code> , <code>GELDING</code> , <code>COVERED</code>
Events can be made non-selectable	<code>event_type.selectable</code> boolean (default true) - allows disabling event types without deletion
Events reference up to 3 horses	<code>horse_primary_id</code> (main subject), <code>horse_secondary_id</code> (e.g. stallion), <code>horse_offspring_id</code> (foal)
Ownership changes track old and new owners	<code>event_old_owner</code> and <code>event_new_owner</code> M:N join tables

File Management Rules

Rule	Enforcement
File paths must be unique	UNIQUE on <code>file.file_path</code>
Files use single-table inheritance	<code>file.file_type</code> discriminator column (varchar(31))

Optimistic Locking

All main entity tables include a **version** integer column (NOT NULL) used for optimistic concurrency control. This prevents lost updates when multiple users edit the same record simultaneously. The application increments the version on each update and rejects writes where the version does not match.

Audit Trail Completeness

Rule	Enforcement
All critical entity changes are audited	Corresponding *_aud table for every business entity
Audit captures who, when, and from where	revision_info stores username, full name, timestamp, and IP address
Collection changes are individually tracked	Separate *_aud tables for M:N joins (horse_owner_aud, horse_breeder_aud, event_new_owner_aud, event_old_owner_aud) and element collections (all horse marking tables)
Audit revisions are ordered chronologically	Index idx_revision_info_custom_timestamp (DESC) for efficient history queries
History can be filtered by user	Index idx_revision_info_username

Search and Fuzzy Matching

The database uses the **pg_trgm** (trigram) extension for fuzzy text search on owner fields:

Index	Table.Column	Purpose
idx_owner_full_name_trgm	owner.full_name	Fuzzy name search
idx_owner_business_number_trgm	owner.business_number	Fuzzy business number search
idx_owner_national_id_number_trgm	owner.national_id_number	Fuzzy ID number search
idx_owner_suffix_trgm	owner.suffix	Fuzzy suffix search

All trigram indexes operate on `lower()` of the column value for case-insensitive matching.

Database Migration & Versioning Strategy

Script Locations

File	Purpose
<code>backend/etc/database/updates.sql</code>	Incremental schema changes applied manually to an existing database (constraint additions, index creation, column alterations, data patches)
<code>backend/etc/database/initial_data/initial_insert.sql</code>	Seed data for a fresh installation (reference data such as countries, event types, etc.)
<code>backend/etc/database/mock_data.sql</code>	Development/test mock data

Deployment Order

The deployment sequence is:

1. Apply any pending entries from `updates.sql` to the target database.
2. Deploy and start the application - Hibernate validates the schema based on entity metadata.
3. Verify application health.

Rollback Strategy

In case of a failed production deployment:

1. Restore the database from the pre-deployment backup (pg_dump snapshot).
2. Redeploy the previous application version.
3. Any data written during the failed deployment window is lost; this must be accounted for in the deployment plan.

Schema Versioning

The `updates.sql` file acts as a chronological log of changes. Individual statements are tracked implicitly by their presence in the file.

Hibernate & Envers Configuration

Base Entity

All main business entities extend `AbstractPkEntity`, which provides:

Feature	Implementation
Primary key	<code>@Id @GeneratedValue(IDENTITY)</code> — database identity column (bigint)
Optimistic locking	<code>@Version int version</code> — incremented on every UPDATE; stale-version writes are rejected by Hibernate
Creation timestamp	<code>@CreationTimestamp LocalDateTime creationDate</code> — set automatically on INSERT
Modification timestamp	<code>@UpdateTimestamp LocalDateTime modificationDate</code> — updated automatically on every UPDATE

Entity-to-Table Mapping Overview

Java Entity	Table	Notes
<code>Horse</code>	<code>horse</code>	Central entity; self-referential FK for sire/dam
<code>Owner</code>	<code>owner</code>	Used for both ownership and breeders
<code>HorseColor</code>	<code>horse_color</code>	Breed-specific reference data
<code>BookType</code>	<code>book_type</code>	Breed-specific studbook sections
<code>Country</code>	<code>country</code>	Reference data; partial-unique index for local flag

Java Entity	Table	Notes
CoveringAgreement	covering_agreement	Stallion breeding season agreements
CoveringCertificate	covering_certificate	Individual mare covering records
CoveringResult	covering_result	Outcome of a covering (1:1 with certificate)
Event / EventType	event / event_type	Horse lifecycle events
LaboratoryTest	laboratory_test	DNA/parentage test records
Season	season	Breeding year periods
User	application_user	Application accounts
RefreshToken	refresh_token	JWT refresh tokens
TokenBlacklist	token_blacklist	Revoked access tokens
AbstractFile / Document / Photo / Attachment	file	Single-table inheritance; discriminator: <code>file_type</code>
CustomRevisionEntity	revision_info	Envers revision master table

Cascade Rules and Orphan Removal

Relationship	Cascade	Orphan Removal	Rationale
Horse → Event (one-to-many)	ALL	YES	Events have no meaning outside a horse; deleting a horse removes all its events
Horse → LaboratoryTest (one-to-many)	ALL	NO	Lab tests follow the horse lifecycle but are not orphan-removed to preserve history

Relationship	Cascade	Orphan Removal	Rationale
Horse → Photo (one-to-many)	ALL	YES	Photos are horse-specific assets
Horse → Attachment (one-to-many)	ALL	YES	Attachments are horse-specific assets
Horse → dnaCard (one-to-one)	ALL	NO	DNA card document follows the horse
Horse → exportCertificate (one-to-one)	ALL	NO	Export certificate follows the horse
Horse → passport (one-to-one)	ALL	NO	Passport document follows the horse
Horse → CoveringAgreement (one-to-many, stallion side)	ALL	YES	Agreements belong to the stallion
Horse → CoveringCertificate (one-to-many, mare side)	ALL	YES	Certificates belong to the mare
Horse ↔ Owner via horse_owner (many-to-many)	none	N/A	Join table managed explicitly; owners are independent entities
Horse ↔ Owner via horse_breeder (many-to-many)	none	N/A	Join table managed explicitly

Fetch Strategies

Association	Fetch	Rationale
Horse.color (→ HorseColor)	EAGER	Color is always displayed alongside the horse;

Association	Fetch	Rationale
<code>Horse.bookType</code> (→ <code>BookType</code>)	EAGER	Studbook type is always shown in horse summaries
<code>Horse.previousBookType</code> (→ <code>BookType</code>)	EAGER	Needed for import/transfer display without extra queries
<code>Horse.father</code> / <code>Horse.mother</code>	LAZY	Pedigree is loaded on demand (passport view, pedigree report)
<code>Horse.owners</code> / <code>Horse.breeders</code>	LAZY	Loaded only when the ownership panel is opened
<code>Horse.events</code>	LAZY	Large collection; loaded only for the event timeline
<code>Horse.laboratoryTest</code>	LAZY	Loaded only for the lab panel; ordered by id DESC
<code>Horse.coveringAgreements</code>	LAZY	Loaded only on stallion detail views
<code>Horse.coveringCertificates</code>	LAZY	Loaded only on mare detail views
All country associations on <code>Horse</code>	LAZY	Loaded on demand; frequently accessed via the <code>vhorse</code> view instead

Custom Hibernate Types and Converters

Converter	Java Type	DB Type	Usage
<code>StringListConverter</code>	<code>List<String></code>	<code>varchar</code> (comma-separated)	Stores small string lists as a single delimited column value; avoids a join table for simple multi-value fields

Envers Configuration

Setting	Value	Effect
<code>store_data_at_delete</code>	<code>true</code>	When an entity is deleted, Envers writes a final audit row preserving the entity's last known state. Without this, the DELETE revision row would contain only null column values
Revision entity	<code>CustomRevisionEntity</code> mapped to <code>revision_info</code>	Extends the standard Envers revision with <code>username</code> , <code>usern_fullname</code> , <code>custom_timestamp</code> (human-readable), and <code>ip_address</code>
Revision listener	<code>CustomRevisionListener</code>	Populates the custom fields from the security context on every audited transaction
Default suffix	<code>_aud</code> (Envers default)	All audit tables follow the <code>{table_name}_aud</code> naming convention
Default revision field	<code>rev</code> (Envers default)	FK column in all <code>_aud</code> tables referencing <code>revision_info.id</code>
Default revtype field	<code>revtype</code> (Envers default)	0 = INSERT, 1 = UPDATE, 2 = DELETE

Performance & Index Strategy

Query Patterns Driving Index Design

Index	Table	Query Pattern
<code>idx_revision_info_customer_timestamp</code> (DESC)	<code>revision_info</code>	History log filtered by date range; timestamp is the primary filter in the <code>vhistory_log</code> view
<code>idx_revision_info_user_name</code>	<code>revision_info</code>	History filtered or joined by user; used when looking up changes made by a specific user
<code>idx_horse_audit_rev_revtype</code> (partial: revtype IS NOT NULL)	<code>horse_audit</code>	History log count queries scanning audit tables; composite (rev, revtype) matches the WHERE clause pattern
<code>idx_laboratory_test_audit_rev_revtype</code>	<code>laboratory_test_audit</code>	Same pattern as above for lab test audit rows
<code>idx_covering_agreement_audit_rev_revtype</code>	<code>covering_agreement_audit</code>	Same pattern for covering agreement audit rows
<code>idx_covering_certificate_audit_rev_revtype</code>	<code>covering_certificate_audit</code>	Same pattern for covering certificate audit rows
<code>idx_owner_audit_rev_revtype</code>	<code>owner_audit</code>	Same pattern for owner audit rows
<code>idx_event_audit_rev_revtype</code>	<code>event_audit</code>	Same pattern for event audit rows
<code>idx_horse_audit_id</code> (partial: id IS NOT NULL)	<code>horse_audit</code>	Horse-specific audit history lookups (fetching all revisions for a single horse)
<code>idx_laboratory_test_audit_horse_id</code>	<code>laboratory_test_audit</code>	Lab test history per horse

Index	Table	Query Pattern
<code>idx_covering_agreement_aud_stallion_id</code>	<code>covering_agreement_aud</code>	Covering agreement history per stallion
<code>idx_covering_certificate_aud_mare_id</code>	<code>covering_certificate_aud</code>	Covering certificate history per mare
<code>idx_event_aud_horse_primary_id</code>	<code>event_aud</code>	Event history per horse (primary subject)
<code>idx_horse_aud_status_registration</code> (partial: <code>status = 'REGISTER_IN_STUDBOOK'</code>)	<code>horse_aud</code>	Studbook registration tracking; filters audit rows where registration occurred
<code>idx_covering_certificate_aud_rev_covering_agreement</code>	<code>covering_certificate_aud</code>	Certificate history lookup by agreement
<code>idx_season_aud_id_rev</code>	<code>season_aud</code>	Season history lookups
<code>idx_application_user_system_role</code> (partial: <code>system_role IS NOT NULL</code>)	<code>application_user</code>	Role-based user filtering (e.g. listing users by role)
<code>idx_application_user_email</code>	<code>application_user</code>	User lookup by email (login, join in history log view)
<code>idx_horse_aud_breed</code>	<code>horse_aud</code>	Breed-scoped audit queries
<code>idx_owner_full_name_trgm</code> (GIN)	<code>owner</code>	Fuzzy owner name search using <code>pg_trgm</code> similarity
<code>idx_owner_business_number_trgm</code> (GIN)	<code>owner</code>	Fuzzy business number search
<code>idx_owner_national_id_number_trgm</code> (GIN)	<code>owner</code>	Fuzzy national ID search
<code>idx_owner_suffix_trgm</code> (GIN)	<code>owner</code>	Fuzzy suffix search

Index	Table	Query Pattern
<code>unique_local_country</code> (partial unique)	<code>country</code>	Enforces exactly one local country; incidentally serves as an index

Tables Expected to Grow Significantly

Table	Growth Driver	Notes
<code>revision_info</code>	Every audited write creates one row	Grows with every user action; the primary audit control table
<code>horse_aud</code>	Every horse INSERT/UPDATE/DELETE	Largest audit table; horses are frequently edited through the registration workflow
<code>event_aud</code>	Every event INSERT/UPDATE	Events accumulate over the lifetime of a horse
<code>covering_certificate_aud</code>	Breeding season activity	Volume spikes during the breeding season (spring/summer)
<code>owner_aud</code>	Ownership changes	Grows with ownership transfer events
<code>laboratory_test_aud</code>	DNA testing activity	Grows with laboratory processing workload
<code>studbook_generation_task</code>	Each studbook generation request	Low-volume but tasks are not automatically pruned

Known Performance-Sensitive Areas

4. **Owner fuzzy search:** GIN trigram indexes (`pg_trgm`) on `owner` are required for acceptable response times on partial-name search across large owner datasets.
5. **Denormalized fields on horse:** `date_of_death`, `date_of_exporting`, and `export_country_id` are annotated `@Denormalized` and derived from events. This avoids joining the `event` table in every horse list query.

Maintenance Considerations

6. **ANALYZE:** `updates.sql` includes explicit `ANALYZE` statements for the most-queried tables (`revision_info`, `application_user`, `horse_aud`,

`laboratory_test_aud`, etc.). These should be re-run after bulk data imports or large migrations.

7. **VACUUM**: PostgreSQL autovacuum handles routine dead-tuple reclamation. Audit tables accumulate inserts with no updates or deletes, so autovacuum overhead is low for them. The `horse` table receives frequent updates (status transitions) and benefits from regular autovacuum activity.
8. **GIN index maintenance**: Trigram GIN indexes on `owner` are updated automatically on INSERT/UPDATE. They do not require manual reindexing unless bloat becomes significant.

Business Rule Enforcement Matrix

The table below classifies each significant business rule by where it is enforced.

#	Business Rule	DB Enforcement	Application Enforcement
1	Only ARABIAN and THOROUGHBRED breeds are valid	CHECK constraint on <code>horse.breed</code> , <code>book_type.horse_breed</code> , <code>horse_color.horse_breed</code> , <code>studbook_generation_task.breed</code>	Enum validation at API layer
2	Horse colors are breed-specific (color must belong to the horse's breed)	UNIQUE(<code>horse_breed</code> , <code>name</code>) and UNIQUE(<code>horse_breed</code> , <code>short_name</code>) on <code>horse_color</code> prevent cross-breed name collisions	Application selects only colors matching the horse's breed in the UI; FK from <code>horse.color_id</code> does not cross-validate breed
3	Color inheritance must be consistent with parents	None	Application checks <code>horse_color.validation_rule</code> (<code>BOTH_PARENTS</code> / <code>AT_LEAST_ONE_PARENT</code>) against parent colors during foal registration

#	Business Rule	DB Enforcement	Application Enforcement
4	Horse name is required	<code>horse.name</code> NOT NULL	Bean validation (<code>@NotBlank</code>) at DTO level
5	Horse sex is required	<code>horse.sex</code> NOT NULL	Bean validation at DTO level
6	Horse status is required	<code>horse.status</code> NOT NULL	State machine enforced in the service layer
7	Horse registration status transitions follow a defined workflow	None (no DB state machine)	Application service layer enforces allowed transitions (e.g. cannot skip from NEW_FOAL to REGISTER_IN_STUDBOOK)
8	Forbidden horse names cannot be used	None	Application queries <code>forbidden_name</code> before persisting; <code>forbidden_name.name</code> has no FK to <code>horse</code>
9	UUID is assigned at creation and immutable	<code>horse.uuid</code> NOT NULL; <code>@Column(updatable=false)</code>	<code>@UuidGenerator</code> assigns on INSERT; <code>updatable=false</code> prevents overwrite
10	UELN is max 15 characters	<code>horse.ueln</code> varchar(15)	Field-length validation at DTO level
11	Transponder code is max 15 characters	<code>horse.transponder_code</code> varchar(15)	Field-length validation at DTO level
12	Name suffix is max 10 characters	<code>horse.name_suffix</code> varchar(10)	Field-length validation at DTO level
13	Horse name is stored in uppercase	None	<code>@PrePersist</code> / <code>@PreUpdate</code> JPA callback converts name

#	Business Rule	DB Enforcement	Application Enforcement
			and nameSuffix to uppercase before every save
14	Pedigree is a directed acyclic graph (no cycles)	None (DB allows a horse to be its own ancestor)	Application must validate pedigree paths when assigning sire/dam
15	A horse has at most one sire and one dam	<code>horse.father_id</code> and <code>horse.mother_id</code> are single-valued FK columns	Enforced by the data model; no multi-sire/dam entry path exists
16	Only one country can be the local (home) country	Partial unique index <code>unique_local_country</code> on <code>country(local)</code> WHERE <code>local = true</code>	UI allows toggling the local flag; DB rejects a second <code>local = true</code> row
17	Country codes follow ISO standards	NOT NULL and UNIQUE on <code>alpha2, alpha3, code</code>	Validation at import/entry time
18	A season year must be unique	UNIQUE on <code>season.year</code>	Application checks for conflicts; DB is the final guard
19	Only one season should be active at a time	None	Application manages the <code>season.active</code> flag; no DB constraint prevents multiple active seasons
20	A stallion has at most one agreement of a given type per season	UNIQUE(type, stallion_id, season_id) on <code>covering_agreement</code>	Application validates before insert; DB is the final guard

#	Business Rule	DB Enforcement	Application Enforcement
21	A covering certificate must reference exactly one stallion source	CHECK constraint <code>exactly_one_stallion_source_check</code> on <code>covering_certificate</code>	Application enforces mutually exclusive selection in the form
22	Agreement number denormalized must equal zero-padded agreement number	CHECK on <code>covering_certificate.agreement_number_denormalized</code>	<code>AgreementNumberFormatter</code> generates the denormalized value; DB validates the format
23	Certificate numbers are unique per agreement	UNIQUE(<code>agreement_number</code> , <code>covering_agreement_id</code> , <code>external_stallion_id</code>)	Application validates sequence within the agreement's number range
24	Each covering has at most one result	UNIQUE on <code>covering_result.covering_certificate_id</code>	Application prevents creating a second result for a certificate
25	Owner identity is unique within country and city	UNIQUE(<code>full_name</code> , <code>country_id</code> , <code>city</code>) on <code>owner</code>	Application checks for duplicate owners; DB is the final guard
26	Owner suffix is globally unique	UNIQUE on <code>owner.suffix</code>	Application generates/validates suffix; DB is the final guard
27	File paths are unique	UNIQUE on <code>file.file_path</code>	Application generates storage paths; DB prevents collision
28	Each horse has at most one DNA card, one passport, one export certificate	UNIQUE on <code>horse.dna_card_id</code> , <code>horse.passport_id</code> ,	Application replaces rather than adds a second document

#	Business Rule	DB Enforcement	Application Enforcement
		<code>horse.export_certificate_id</code>	
29	Document type is within the valid range (0–2)	CHECK on <code>file.document_type</code> BETWEEN 0 AND 2	Enum mapping at application level
30	Refresh tokens are unique	UNIQUE on <code>refresh_token.token</code>	Token generated with a secure random generator; collision probability negligible
31	Access token blacklist uses unique tokens	UNIQUE on <code>token_blacklist.token_value</code>	Application adds to blacklist on logout; DB prevents duplicate entries
32	User account is unique per email + breed	UNIQUE(email, breed) on <code>application_user</code>	Application validates before creation; DB is the final guard
33	Users are soft-deleted, never hard-deleted	<code>application_user.deleted</code> boolean default false	Application sets <code>deleted = true</code> instead of issuing DELETE; no DB trigger enforces this
34	Optimistic concurrency — no lost updates	@Version int <code>version</code> column on all main entities; Hibernate throws <code>OptimisticLockException</code> on version mismatch	Application catches the exception and returns a conflict response to the client
35	Lab users must be associated with a laboratory	Application validates that LAB_USER role accounts have a non-null <code>laboratory_id</code>	<code>application_user.laboratory_id</code> is nullable at DB level

#	Business Rule	DB Enforcement	Application Enforcement
36	Audit captures every change to business entities	Hibernate Envers <code>@Audited</code> on all core entities triggers automatic <code>_aud</code> row insertion	Auditing is transparent; application code does not explicitly write audit records

All trigram indexes operate on `lower()` of the column value for case-insensitive matching.

Known Technical Limitations & Scaling Considerations

Schema Constraints That May Require Refactoring at Scale

file table uses **single-table inheritance**. All file subtypes (`Document`, `Photo`, `Attachment`) share one table with a `file_type` discriminator. Columns specific to one subtype are null for others. This is acceptable at current scale but means the table will accumulate many nullable columns if new file subtypes are added. A joined-table or table-per-class inheritance strategy would be cleaner at scale.

Anticipated Growth Considerations

Area	Growth Pattern	Threshold to Watch
<code>studbook_generation_task</code>	One row per generation request	No cleanup; will grow until pruned manually
<code>file</code>	Driven by passport scans, DNA cards, photos	Storage-side concern (filesystem); the DB row itself is small, but the single-table design will require attention if many new file types are added